The Life Cipher: Authenticated Encryption from Conway's Game of Life with Mined Keys

Abstract

We describe the Life Cipher, an experimental authenticated encryption scheme with associated data (AEAD) built on Conway's Game of Life on a 32×32 torus. Its 1,024-bit key is itself a Life configuration, found by random search because it runs through at least 6,000 distinct generations before repeating. In one mining run, about one random grid in 43 million qualified. Key setup grows the key's first 6,000 generations and runs the cipher on them to build a secret table of round keys, each the XOR of 30 shuffled generations. The state is two grids updated by second-order Life, $(A, B) \leftarrow (B, \mathrm{Life}(B) \oplus A \oplus RK)$, which is a permutation for every key. The state drives a duplex mode much like Ascon's. Round keys are picked by a nonlinear function of the state, and the C implementation is designed to run in constant time. We report our own analysis: a statistical test harness calibrated with controls of known quality, and attacks on reduced rounds. We choose 24 rounds per block using a heuristic margin over reduced-round results. Extrapolating the measured cell persistence suggests a distinguisher through about seven rounds within a limit of 2^{48} blocks per key; we have not demonstrated that attack at seven rounds or established a security bound at 24. Nothing about the cipher is proven, and nobody outside the project has analyzed it. We publish the specification, three implementations, test vectors, and a reduced-round key-recovery challenge, to invite that analysis.

Keywords: authenticated encryption, cellular automata, Game of Life, duplex construction, symmetric cipher design, reduced-round cryptanalysis, cryptanalysis challenge, statistical testing

1 Introduction

Conway's Game of Life [Gar70] turns a simple local rule into behavior complex enough to compute anything [BCG82]. Cellular automata have tempted cipher designers for as long as they have been studied. Wolfram proposed Rule 30, a one-dimensional automaton, as a stream cipher generator [Wol86a, Wol86b]. It passed the statistical tests of its day, and Meier and Staffelbach then recovered its keys from known keystream [MS91]. That history sets the tone for this paper: statistical tests show little, and a new design needs analysis. Schneier makes the broader case for subjecting new ciphers to experienced, independent cryptanalysis [Sch98].

This paper asks a narrow question. Can a modern AEAD be built around the Game of Life itself, rather than around a cellular automaton chosen for its algebra, and how does such a design fare against standard analysis? The Life Cipher answers the first half with a concrete design. The key is a 32×32 Life configuration mined for its long life. The round keys are generations of that configuration, combined and selected by the cipher's

This paper describes the Life Cipher as specified in `SPEC.md` draft 0.4, algorithm version 1. The specification, both implementations, the test vectors, a description of the test harness and the challenge are at https://github.com/t3spe/life-cipher. A command-line tool for encrypting files with the cipher, built on a third implementation in Go, is at https://github.com/t3spe/life-cipher-runner.

This work is licensed under a "CC BY 4.0" license.
Date of this document: 2026-11-11.

state. The state is updated by second-order Life, which makes each round invertible. For the second half, we report our measurements.

Contributions.

- **A complete design:** an AEAD with a 1,024-bit key, a 1,024-bit IV and a 128-bit tag, specified precisely enough to implement (Section 3), with a stated rationale (Section 4).
- **Implementations and test vectors:** a Python reference implementation; a constant-time C implementation with AVX2 and portable builds; and a Go implementation with a command-line tool, `lce`, that encrypts files. All three agree with three published vector files (Section 6).
- **A validation method:** a test harness of 48 checks in three tiers, calibrated with 17 controls, and a heuristic minimum for the number of rounds, $R \geq 3\max(r_{\min}, r_{\text{attack}})$, compared with ChaCha as a heuristic check (Section 5.2).
- **A reduced-round challenge:** key recovery at 4, 8, 12, 16 and 24 rounds, with 2^{20} blocks of known plaintext per tier and salted commitments to the keys (Section 7).

What we don't claim. This is an experiment. Nothing about the cipher's security is proven. The analysis is our own, and no one outside the project has checked it. The cipher must not protect real data. Our statistical results show that the outputs don't fail the tests we ran, and nothing more.

Outline. Section 2 gives background. Section 3 specifies the cipher, and Section 4 explains its choices. Section 5 analyzes its security, Section 6 its implementations, and Section 7 describes the challenge. Section 8 concludes.

2 Background

2.1 Life on a torus

A Life configuration is a grid of cells, each live or dead. One step, written Life(X), applies rule B3/S23 to every cell at once: a dead cell with exactly three live neighbors becomes live, a live cell with two or three live neighbors stays live, and every other cell is dead in the next generation. The neighbors of a cell are the eight cells around it. We use a 32×32 grid whose edges wrap, a torus, so every cell has eight neighbors.

Life isn't reversible. Many configurations have no predecessor, and many pairs of configurations share a successor. On a finite torus, every configuration's generations eventually repeat, since there are only 2^{1024} grids. Random configurations usually settle within a few hundred or thousand generations into still lifes and short-period oscillators. We write $D(K)$ for the number of distinct generations of K before the first repeat: $\text{Life}^0(K), ..., \text{Life}^{D-1}(K)$ are all different, and $\text{Life}^D(K)$ equals one of them.

2.2 Long-lived configurations

Life's literature calls a small pattern that takes a long time to settle a methuselah. The R-pentomino, five cells on the infinite plane, settles only after 1,103 generations [LW]. Our keys are long-lived in the same sense, but they aren't small patterns. They're random fields of 64 to 960 live cells that happen to run through at least 6,000 distinct generations on the 32×32 torus. Finding them takes search: our mining programs, the miners, draw random grids and run each one until it repeats. In one mining run, about one random grid in 43 million qualified. The pool behind this paper holds 387,874 vetted keys.

2.3 Second-order reversible automata

Any update rule f, reversible or not, gives a reversible automaton on pairs of configurations: $(X_{t-1}, X_t) \mapsto (X_t, f(X_t) \oplus X_{t-1})$. Given the new pair, the old one comes back as $X_{t-1} = X_{t+1} \oplus f(X_t)$. This second-order construction is due to Fredkin, and Toffoli and Margolus describe it [TM87]. Cryptographers will recognize it as a balanced Feistel round on two halves, with round function f. The Life Cipher's round is this construction with $f(B) = \mathrm{Life}(B) \oplus RK$, where the round key RK itself depends on B (Section 3.3).

2.4 The duplex construction

A duplex [BDPV11] keeps a state larger than its data blocks. It absorbs each block into an outer part of the state, applies a permutation, and reads the outer part out as keystream. The inner part, the capacity, is never read or written directly, and generic attacks on the construction cost about $2^{c/2}$ for a capacity of c bits. Ascon, standardized by NIST in SP 800-232 [NIST25], is a duplex AEAD built on a fixed 320-bit permutation, with a keyed initialization and finalization.

The Life Cipher has the same overall shape. Its state is two 1,024-bit grids. Data goes into grid B, the pad is read from grid B, and grid A plays the part of the capacity. The difference is the permutation: here it's keyed, and its round keys are picked by the state. Existing duplex security analyses, including those for full-state keyed duplexes [MRV15, DMV17], model the underlying permutation as a public random permutation and introduce the key through the construction's state. Here, the key determines the round-key table and therefore the permutation itself. Those results do not directly establish security for this construction.

2.5 Cellular automata in cryptography

Besides Rule 30 [Wol86a, Wol86b] and its cryptanalysis [MS91], cellular automata appear in cryptography in many forms. Nandi, Kar and Pal Chaudhuri proposed ciphers from programmable automata [NKP94], which Blackburn, Murphy and Paterson showed to be affine and easy to break [BMP97]. Daemen's thesis studied invertible nonlinear automata as building blocks [Dae95]; the χ step of Keccak descends from that work. Life-like automata have been used in image encryption and chaotic schemes, for example by Machicao, Marco and Bruno [MMB11], usually without analysis against modern attacks.

The Life Cipher differs from these in two ways. It uses Life unchanged, rule B3/S23, rather than an automaton chosen for its algebra. And it treats the automaton as one part of a conventional design, a keyed permutation in a duplex mode, which it can then be measured against.

3 The Life Cipher

This section condenses the specification (`SPEC.md` draft 0.4). The specification is normative, and where the two differ, it wins.

3.1 Notation and data

A **grid** is a 32×32 array of cells on a torus, each 0 (dead) or 1 (live). Cell (r, c) is in row r and column c, both counted from 0 at the top left and taken mod 32.

- **Bytes.** A grid corresponds to 128 bytes: cell (r, c) is bit $7 - (c \bmod 8)$ of byte $4r + \lfloor c/8 \rfloor$, where bit 0 is the least significant.
- **Row words.** $X[r]$ is row r of grid X as a 32-bit integer, its four bytes read big-endian, so column c is bit $31 - c$.

- **Operations.** $\oplus$ is XOR, $\mathrm{rotl}(x, n)$ rotates a 32-bit word left by n, and $\mathrm{shift}(X, dx, dy)$ moves grid X by dx columns right and dy rows down, with wrapping: $Y(r, c) = X(r - dy, c - dx)$.
- **The fold.** $F(X) = \bigoplus_{r=0}^{31} \mathrm{rotl}(X[r], r)$ maps a grid to a 32-bit word. It's linear over GF(2).

3.2 Parameters

Name	Meaning	Recommended value
K	the key, a vetted grid (Section 3.6)	1,024 bits
IV	never repeated under one key	1,024 bits
τ	the tag	128 bits
T	generations in the key table	6,000
S	entries per group in the round-key lookup	16
k	generations XORed into each round key, and the number of shuffles	30
R	rounds after each block	24
$R_{\text{init}}, R_{\text{final}}$	rounds at the start and before the tag	$2R = 48$
version	algorithm version	1

Constraints: T must be a positive multiple of S and of k; S must be 16, because the selector produces a 4-bit index; and the specified parameter sets require $R \geq 16$; smaller values are used only for analysis and the challenge. Life alone propagates a difference by at most one cell per round, but the state-dependent selector can change the round key across the whole grid in one round. Local propagation therefore does not establish a lower bound on R.

Data limit. Each key may process at most 2^{48} blocks, 2^{55} bytes when the blocks are full. Every block the cipher absorbs counts, in encryption and decryption, including each message's last block with the end marker, so even an empty message counts one. Section 5.3 explains the limit.

A lighter demo set, with $k = 4$ and $R = 16$, exists for teaching. It isn't a recommendation.

3.3 The round function

Selectors. Each round needs a public group g, one of $T/S = 375$, and 14 secret bits: an entry j within the group and a shift (dx, dy). They're computed from grid B, the round counter r and an IV offset o, by rule v5:

$$h \leftarrow r \cdot \texttt{0x9E3779B9} \bmod 2^{32},$$

then for $i = 0, ..., 31$:

$$h \leftarrow \mathrm{rotl}(h \oplus B[i], 13) \cdot \texttt{0x9E3779B9} \bmod 2^{32}.$$

With $s = \lfloor h/2^{18} \rfloor$, the top 14 bits of h: $j = s \bmod 16$, $dx = \lfloor s/16 \rfloor \bmod 32$, $dy = \lfloor s/512 \rfloor$, and $g = (o + r) \bmod 375$. The multiplier is $\lfloor 2^{32}/\varphi \rfloor$. The IV offset is $o = F(\mathrm{IV}) \bmod 375$ for messages, and 0 during key setup.

The Mix round. On state (A, B, r), with a table Tab of T grids:

1. Compute (g, j, dx, dy) from B, r and o.
2. $RK \leftarrow \mathrm{shift}(\mathrm{Tab}[16g + j], dx, dy)$.
3. $(A, B) \leftarrow (B, \mathrm{Life}(B) \oplus A \oplus RK)$.
4. $r \leftarrow r + 1$.

Mode 1 uses the setup table H and runs only during key setup. Mode 2 uses the round-key table W and processes all messages. Reading $\mathrm{Tab}[16g + j]$ must not reveal j: implementations read all 16 entries of group g and keep one with masks.

The selectors depend only on B, r and o, and B survives the round as the new A. So from the state after a round, the selectors, the round key and then the old $A = B' \oplus \mathrm{Life}(A') \oplus RK$ can all be recomputed. For fixed round counter and IV offset, each round is a permutation of the 2,048-bit state for every key: states never merge. Reversibility alone does not establish the density or statistical properties of the grids.

3.4 Key setup

Key setup turns K into the key state (K, W, M), or fails.

1. **Key table:** $G[t] = \mathrm{Life}^t(K)$ for $t = 0, ..., T - 1$.
2. **Key check:** fail unless $G[T - 1]$ differs from each of $G[0], ..., G[T - 2]$. Because Life is deterministic, a repeat anywhere in the first T generations makes $G[T - 1]$ a repeat too, so this check passes exactly when $D(K) \geq T$. It takes 5,999 comparisons, run in full with no early exit. A vetted key always passes.
3. **Setup table:** $H[i] = \bigoplus_{m=0}^{k-1} G[(i + mT/k) \bmod T]$ for $i = 0, ..., T - 1$.
4. **Setup constant:** SETUP is the first 1,024 bits of π's fractional part (π = 3.243F6A88… in hexadecimal), as a grid, with T, k, S, R and the version XORed into its row words 0 to 4.
5. **Setup run:** set $A \leftarrow$ SETUP, $B \leftarrow K$, $r \leftarrow 0$, $o \leftarrow 0$. Run R_{init} rounds in mode 1 and output $Z_0 = B$. Then, for $i = 1, ..., kT/16$, run R rounds and output $Z_i = B$.
6. **Mask:** $M = Z_0$.
7. **Tags and shuffles:** split $Z_1 \parallel Z_2 \parallel \cdots$ into kT 64-bit big-endian tags. Shuffle π_s, for $s = 0, ..., k - 1$, is $0, ..., T - 1$ sorted by tag $sT + i$, with ties going to the smaller i.
8. **Round-key table:** $W[s \cdot T/k + q] = \bigoplus_{m=0}^{k-1} G[\pi_s[kq + m]]$ for $s = 0, ..., k - 1$ and $q = 0, ..., T/k - 1$.

W has T entries, and every generation appears in exactly k of them, once per shuffle and never twice in one entry. The sort must not reveal the tags, so implementations use a fixed sorting network. They also must not read G at a position that depends on a shuffle. Carrying each generation through the network along with its tag does both.

3.5 Encryption and decryption

Encryption takes the key state, a 128-byte IV, associated data AD and plaintext P, and returns a ciphertext C of exactly $|P|$ bytes and a 16-byte tag τ. All rounds are in mode 2.

1. **Init:** $A \leftarrow \mathrm{IV} \oplus M$, $B \leftarrow K$, $r \leftarrow 0$, $o \leftarrow F(\mathrm{IV}) \bmod 375$. Run R_{init} rounds.
2. **Associated data,** if there is any: in blocks of up to 128 bytes, XOR each block into the first bytes of B. After the last block, of $L < 128$ bytes, XOR 0x80 into byte L. Run R rounds after every block. If $|AD|$ is a multiple of 128, a final empty block carries only the 0x80.
3. **Separator:** flip cell (31, 31) of A. This happens even when there's no AD.
4. **Message:** in blocks of up to 128 bytes. For a block of L bytes, the pad is the first L bytes of B, the ciphertext block is the plaintext block XOR the pad, and the ciphertext block replaces those L bytes of B. After the last block, of $L < 128$ bytes, XOR 0x80 into byte L. Otherwise run R rounds and go on. An empty message, or one whose length is a multiple of 128, ends with an empty block that carries only the 0x80.
5. **Finish:** run R_{final} rounds. The tag is the first 16 bytes of B.

Decryption runs the same steps, with the plaintext block computed as the ciphertext block XOR the pad, and the received ciphertext absorbed. It compares the computed tag

with the received one over all 16 bytes in constant time, and if they differ, it returns FAIL and releases no plaintext.

3.6 Key vetting

Keys are vetted once, offline, before use. A key passes if:

1. $D(K) \geq 6,000$, the minimum generations, which must be at least T;
2. its complexity score is at least 10,000; and
3. it has between 64 and 960 live cells.

The complexity score. It comes from the miners' scorer, and the specification defines it formally. Over the key's $N = D + 1$ generations:

- H_{pop} is the entropy of the population count over 32 bins, normalized to $[0, 1]$.
- H_{cell} is the mean binary entropy of each cell's share of time alive.
- I is one minus the mean absolute correlation between pairs of cells.

The score is $\lfloor H_{\text{pop}} \cdot H_{\text{cell}} \cdot I \cdot \ln N \cdot 10,000 \rfloor$. The specification fixes the details the scorer's code implies, such as how pairs with a cell that never changes are counted. An independent implementation written from that text agrees with the scorer on 2,000 sampled pool keys and on every test field.

Why three tests. D alone can be inflated without real complexity. A field built from oscillators and spaceships with coprime periods doesn't repeat until the least common multiple of those periods, and can have D in the millions while every cell's history is trivial. The ten crafted fields we tried score 181 to 3,273, while every pool key with $D \geq 6,000$ scores 11,107 or more. Before vetting, there are $\binom{1024}{n}$ grids with n live cells: at least 2^{341} over the permitted range, and about 2^{994} at the pool's median of 418. Vetting restricts this set. These counts describe the candidate space, not the entropy of accepted keys or a lower bound on key-recovery work; we have not bounded the accepted set at each population.

Key generation. The security goals assume keys made as the miners make them: a density drawn uniformly from 1–99%, then that many live cells placed uniformly at random, from a strong random source. Each key must come from its own draw and remain secret. A published collection of mined configurations is suitable for experiments, not for choosing operational keys: selecting from a known list limits security to searching that list.

4 Design rationale

The key is the configuration itself. It's the simplest choice, and mining already produces such keys. Our mining procedure identifies valid keys by evolving candidate grids. One classifier on counts of local 3×3 patterns scored 50.5% accuracy (AUC 0.507) when distinguishing mined keys from random grids with the same live-cell counts. This tests one feature set and classifier; it does not rule out cheaper recognition or enumeration methods.

Vetting offline, plus one cheap check. The complexity score is costly, and its running time depends on the key, so it runs offline, once per key. Setup checks only that the first T generations are distinct. That check costs 5,999 grid comparisons in constant time, on a table setup computes anyway, and it stops the likeliest misuse: random bytes used as a key. Random grids mostly fall into short cycles, after which their table collapses; in our tests, such keystreams failed statistical tests within a megabyte.

A duplex on second-order Life. Plain Life loses information at every step, so a state run under it would merge and thin out. The second-order form is reversible, and two grids are the least it needs. The hidden grid A has 1,024 bits, corresponding to a 2^{512}

capacity benchmark in an idealized duplex model. As Section 2.4 explains, this is not an established security bound for our keyed, round-dependent construction.

Round keys from a shuffled table of generations. Each round key is the XOR of k generations chosen by a key-dependent shuffle. Fixed spacing, such as $G[t], G[t+100], ...$, was considered and rejected as predictable. Generations about 100 or more apart are about as different as two unrelated grids, even at the best torus shift, which supports mixing generations from separated times. This distance measurement does not establish independence: every generation is a deterministic function of K. The table is built once per key, so each round costs one lookup.

Dense round keys. A generation is only about 9% live. A round key that sparse can't hide Life's bias, and cells then persist from block to block (Section 5.3). XORing k generations raises the density: about 27% at $k = 4$, 39% at $k = 8$, 47% at $k = 16$, and 49.7% at $k = 30$. The setup table H does the same for key setup, with fixed positions and no secret.

A public group with a secret pick. A table lookup at a secret index leaks through the cache, as table-based AES implementations showed. So the group of 16 entries is public, picked by the IV offset and the round counter, and only the pick within it and the shift are secret. The cost is 2 KB of reads per round and 14 secret bits per round.

Selectors from B only, through a nonlinear chain. Reading only B keeps Mix reversible. The cost is one public choice per full block: right after a full ciphertext block is absorbed, B is that ciphertext, so the next round's selectors are public, although the chosen entry's contents stay secret. An earlier rule read B through the linear fold F, and a difference that cancels in F and that Life ignores slipped past it. The chain of rule v5 sees every change in B, except when its 14 output bits collide.

A 1,024-bit IV masked by M. An IV only has to be unique, and a full grid reaches every cell from round 0. The secret mask M stops an attacker who chooses IVs from setting A, and keeps the whole state secret from the start. It also means no message can reproduce the setup run.

The end marker and the separator. Together they make every (AD, message) pair absorb a different sequence of blocks. Without the marker, trailing zero bytes could be cut unnoticed. Without the separator, bytes could move across the AD/message boundary: AD = X with a message of one block of 0x80 and zeros, and AD = X followed by that block with an empty message, would absorb the same blocks.

T = 6,000. Past 6,000 generations, every extra 234 or so halves the pool of usable keys: a minimum of 7,000 would keep 5.5% of them. Below it, nothing is gained, because the table stays busy to its end: about 70 cells change from one generation to the next, and activity drops only in the last 100 or so generations before a key repeats.

Constants. SETUP is π's first 1,024 bits, a rule that leaves no room for choice, as in Blowfish [Sch94]. Its parameter words keep different settings and versions from sharing a setup run. The multiplier $\lfloor 2^{32}/\varphi \rfloor$ is a familiar odd constant. The rotation 13 was the value first measured for rule v5, and kept. The 0x80 marker appends a one bit followed by zeros under our bit convention. Standardized Ascon uses the same padding principle, encoded as 0x01 under its little-endian convention [NIST25].

5 Security analysis

5.1 Goals and threat model

These are design goals, not proven claims:

Property	Goal
Key recovery	at least 2^{256} work
Distinguishing ciphertexts from random, under adaptive chosen plaintexts and chosen IVs that never repeat	at least 2^{256} work
Integrity of the IV, AD, ciphertext and tag	a forgery succeeds with probability about 2^{-128} per attempt
Idealized capacity benchmark	2^{512} for a 1,024-bit capacity; no bound established for this construction
Data per key	at most 2^{48} blocks

The attacker knows the algorithm, sees every IV, AD, ciphertext and tag, may choose plaintexts and IVs, may submit forgeries, and may run code on the same machine and observe cache use and timing. Physical attacks are out of scope.

Related keys: no goal. The goals assume independent keys. This matters more here than for most ciphers. Life commutes with the torus's 8,192 symmetries (1,024 shifts times 8 rotations and reflections). So a transformed key $\sigma(K)$ has the transformed table $\sigma(G)$, and the symmetry orbit of a vetted key contains at most 8,192 distinct grids, all of which pass vetting. There are fewer if a nonidentity symmetry fixes the key. Only the setup constant, the selector chain and the IV handling break the symmetry. Keys must never be derived from one another by a symmetry or by editing cells. Independent keys aren't scarce: the pool holds 387,874, and no two of them have the same next generation, even allowing for symmetry. AES-256 has a related-key attack at about $2^{99.5}$ work [BK09] and remains in wide use, because protocols don't create related keys. Our harness still checks how the key is mixed in, with one-cell flips and with symmetries, and both pass, but that's evidence about the design, not a claim.

IV reuse. There's no goal when an IV repeats. Two messages under the same key, IV and AD get the same pads up to and including their first differing block, which then leaks the XOR of the two plaintexts.

5.2 Method

The harness. We tested the cipher as a black box, and in places as a white box, with a harness of 48 checks in these families:

- **correctness (T):** round trips, determinism, lengths, the key-vetting tool, and Mix run backwards
- **forgery and tampering (F):** bit flips, truncation, moving bytes across the AD/message boundary, block reordering and splicing, and random tags
- **statistical streams (S):** ten streams fed to PractRand [Dot] and TestU01 [LS07]:
  - one long message
  - many short ones
  - counter IVs
  - key, IV and plaintext avalanche
  - special IVs
  - plaintext XOR ciphertext
  - torus-symmetric related keys

    - the setup run's own output
- **avalanche matrices (A)**
- **grid-aware statistics (G):** among them G7, which compares each cell with the same cell one and two blocks later over a whole long stream
- **collisions (C)**
- **predictability (P):** linear complexity, linear prediction, a small convolutional network able to represent any local rule including Life, and prediction across consecutive IVs
- **setup (U):** shuffle uniformity, W composition, and independence of M and K
- **constant time (K):** a dudect-style timing test, K1 [RBV17], and Valgrind secret tracking, K2, in the manner of ctgrind [Lan10]
- **misuse (M):** informational only

The tiers are smoke (under a minute), standard (each stream to 1 GB, with SmallCrush) and deep (three streams to 256 GB and the rest to 4–16 GB, with Crush and BigCrush, and G7 on 64 GB).

Controls. Seventeen controls calibrate the harness. ChaCha20-Poly1305 [RFC8439] and Ascon-AEAD128 must pass everything. Deliberately flawed controls must fail specific tests. Among them are a linear feedback shift register, the Mersenne Twister, ChaCha20 with a 20-bit key, and versions of the Life Cipher with one of its own decisions undone: no end marker, no separator, a secret table index, a selector that also reads A, no shuffles, a raw Life keystream, and two rounds per block. Every graded test except the vetting tool's must be failed by at least one control, so no test is decoration. A result is a FAIL at $p < 10^{-6}$, or on any accepted forgery or K2 finding. Results between 10^{-6} and 10^{-3} are rerun, at most twice, with a new seed and four times the data. A TestU01 battery gives one result: its smallest two-sided p-value, corrected for the number of p-values it reports (Bonferroni).

The margin rule. We choose $R \geq 3\max(r_{\min}, r_{\text{attack}})$. Here $r_{\min}$ is the smallest tested R at which every graded test passes for the stated tier and sample sizes, and r_{attack} is the greatest attack reach within a stated data budget, distinguishing demonstrated results from model estimates. The factor 3 is a heuristic allowance for attacks the harness misses, not a bound on their reach. As a check on its scale, we compared it with ChaCha using a variable number of rounds. The harness alone breaks ChaCha at 3 rounds and passes it at 4, at small samples and at full size alike, so its $r_{\min}$ is 4. Published attacks reach about 7 rounds [AFK+08]. The heuristic minimum is then $3\max(4, 7) = 21$, close to the 20 rounds ChaCha20 uses. This single comparison does not validate the factor for a different design. It illustrates why statistical tests alone cannot set a round count. For Life, we report demonstrated attacks and extrapolated estimates separately and use the latter in the round-selection heuristic.

5.3 The margin for the recommended set

The recommended set combines rule v5, $k = 30$ and the setup table H.

Cell persistence. Write X_t for grid B after round t. Each round gives $X_{t+1} \oplus X_{t-1} = \text{Life}(X_t) \oplus RK_t$. Life's output is about 27% live: its correlation with the constant 0 is 0.453, the largest coefficient in its Walsh spectrum. Where the round keys are sparse, the right-hand side is then mostly 0, and a cell tends to agree with itself two rounds later. With R even, the pads of consecutive blocks, which a known-plaintext attacker sees in full, are linked cell by cell through $R/2$ such terms. At $k = 30$ the round keys are 49.7% live. The correlation between each cell and the same cell one block later averages 0.0029 at $R = 2$ over 32 keys, with 0.0045 for the weakest sampled key, and is about $8 \cdot 10^{-6}$ at $R = 4$ ($z = 4.4$, on 32 GB pooled over 8 keys). It falls about 4.2 bits per round between $R = 2$ and 4.

Chosen-IV differentials. Two IVs that differ in cells (r, c) and $(r + d, c + d)$, on one wrapped diagonal, have the same fold, and so the same IV offset and round-key groups. With round keys as dense as the recommended set's, none of 160 million such pairs over 32 keys gave first output blocks at $R = 2$ that differ in fewer than 400 of 1,024 bits, which random outputs do with probability about $8.4 \cdot 10^{-13}$. Under an independent-trial model, that bounds the rate below $1.9 \cdot 10^{-8}$ (one-sided, 95%); it is neither an attack rate nor a bound for every vetted key, and it sets no round limit.

- $\boldsymbol{r_{\min} \leq 6}$**.** This design passes all 71 standard-tier results at $R = 6$. Smaller R wasn't run at full size.
- **Extrapolated $\boldsymbol{r_{\text{attack}} \approx 7}$ within $\boldsymbol{2^{48}}$ blocks per key.** The strongest attack we have at $k = 30$ is cell persistence. Extrapolated from its decay between $R = 2$ and 4, a key's 2^{48} blocks could detect it up to about $R = 6$ for an average sampled key and 7 for the weakest sampled key. These are estimates, not demonstrated attacks at those rounds or bounds over all vetted keys.
- $\boldsymbol{R = 24}$**.** The heuristic asks for at least $3 \times 7 = 21$. We chose 24, about 3.4 times the estimated attack reach for the weakest sampled key. This accommodates an estimate of eight rounds under the same rule, but does not establish a security margin against unmeasured attacks or unsampled keys.

The estimate depends on the data budget. Under an approximate independent-sample model, the block count for a z-standard-deviation detection of correlation c is $N_z \approx z^2/(1024c^2)$. For $z = 1$, the measured $c(4) \approx 8 \cdot 10^{-6}$ and an assumed decay of 4.2 correlation bits per round give

$$\log_2 N_z(R) \approx 23.9 + 8.4(R - 4) + 2 \log_2 z.$$

At $z = 1$, this model puts a 2^{64}-block detection threshold near nine rounds and a 2^{256} -block threshold near 32 rounds, before applying the margin heuristic. The latter is a data-complexity estimate, not a demonstration of 256-bit security. It also differs from requiring the correlation itself to fall below 2^{-256}. The fit uses even round counts. It does not establish the decay at odd round counts or reproduce the separate estimate for the weakest sampled key. The seven-round estimate above remains provisional pending those derivations. A four-standard-deviation threshold adds four bits to the data requirement. All these estimates extend a fit from two and four rounds far beyond the observations; they may miss slower decay, other relations, or weaker keys.

The specified limit is 2^{48} blocks per key, counting every block absorbed in encryption and decryption, including each message's last block with the end marker. Every message, even an empty one, and every verification attempt therefore counts at least one block, so the limit also bounds initializations, tags and forgery attempts. We have not established a security bound covering those query counts. The limit and round count are design choices motivated by the measured attack, not sufficient conditions for the goals in Section 5.1.

5.4 The final runs

On the recommended set, at $R = 24$:

- **The standard tier,** run with every control, passes all 71 graded results (30 October 2026).
- **The recorded deep-tier summary reports 72 passes** (1 November 2026, about 36 hours on 14 worker processes):
  - PractRand stays clean to 256 GB on one long message, byte-swapped too, on the counter-IV stream, and on the setup stream across 190,872 keys.
  - TestU01′s Crush passes on all three. BigCrush on the long message gives 0 of 160 p-values outside $[0.001, 0.999]$.
  - G7 on 64 GB finds a correlation of $2.19 \cdot 10^{-6}$ at lag 1 and $-1.51 \cdot 10^{-6}$ at lag 2 ($p = 0.21$).

  - None of 10^8 random tags was accepted.
  - The deep run's one FAIL, on the setup stream's Crush, was a harness bug: the battery's key list covered only 1 GB of stream, so Crush saw the same data about 120 times. With the fix, the rerun has 1 of 144 p-values outside $[0.001, 0.999]$, the worst $p = 0.0008$. Corrected for the 144 p-values, as Section 5.2 grades a battery, that's $p = 0.23$, a pass.
- **Draft 0.4′s key check** changes no output of a vetted key. Its standard tier passes all 71 again (5 November 2026).

Constant time.

- **K1** compares the timing of setup, encryption and decryption under fixed and random secrets. It finds no leak: the largest $|t|$ is 2.52 in the deep run, and at most 1.16 in the draft 0.4 run, against a threshold of 4.5.
- **K2** runs the implementation under Valgrind with the key and plaintext marked as undefined, and finds no branch or memory address that depends on them, on either build.
- **K2 does see the key check.** A deliberately broken build that exits the check early fails K2 at that loop.
- **The controls show both tests work.** The control with a secret table index fails K1 ($|t| = 407$) and K2.

5.5 What the tests can't show

Passing statistical tests is weak evidence. Rule 30 passed the tests of its time, and fell to an attack those tests couldn't see [MS91]. Our harness found cell persistence only indirectly: the persistence test existed in G5, but at a data size where it was blind, and PractRand passed streams that a dedicated test failed. The comparison with ChaCha shows the gap plainly: the harness breaks 3 rounds, and published cryptanalysis 7. Our margin rests on one extrapolation, the persistence decay from $R \leq 4$, and on the attacks we thought of. At one round the cipher is broken outright: with known plaintext, each block reveals one table entry, unshifted, since that round's selectors come from public values. Beyond one round, we did not attempt algebraic attacks, including SAT-based state or key recovery. The 6.1 Mbit table and the hidden states are all constrained by one 1,024-bit key and deterministic transitions; their size does not establish attack difficulty, and each block does not introduce fresh secret entropy. Experiments on a smaller torus and table could help develop such attacks before testing the full design. Related keys and multiple users are outside our goals.

5.6 Properties to watch and open problems

- **The linear fold now sets only the IV offset.** Some simple IVs fold to exactly 0, for example the counting bytes 00 01 … 7F.
- **Public selectors.** In the round after each full ciphertext block, the selectors are public.
- **The 14 secret selector bits** cap what one round contributes against differentials in which Life ignores a difference.
- **Keys whose table is sparser than average** are weaker against persistence.
- **Multi-cell linear relations,** such as a cell against the parity of its neighbors, Life's next-largest Walsh coefficient at 0.328, are untried.
- **Exact linear relations among W entries.** Each shuffle partitions all T generations, so the $T/k = 200$ entries made from one shuffle XOR to the same grid: the XOR of all generations. At $k = 30$ that gives 29 exact relations.
- **The setup table repeats.** By its definition, $H[i]$ depends only on $i \bmod T/k$, so H has at most 200 distinct entries, each appearing 30 times, and the setup run's round

keys are shifts of those 200 grids. The setup stream's tests (Section 5.4) found no effect, but the structure is there to exploit.

- **Generations determine their successors.** Any one generation $G[t]$ gives every later generation by running Life forward. Earlier generations need predecessor search. The Life community's SAT-based tools do it [LLS, GS], but we haven't measured how they fare on full 32×32 fields. How much of the table a partial recovery would expose is open.
- **Sharper chosen-IV distinguishers,** using the whole distribution of difference weights rather than a threshold, are untried.

6 Implementation and performance

Reference. The Python reference implementation mirrors the specification section by section, makes no attempt at constant time, and says so. It wrote the test vectors.

Constant-time C. The C implementation, C11 with GCC and Clang extensions, has an AVX2 build and a portable build. No secret decides a branch or a memory address:

- Life is bit-parallel on the 32 row words.
- The table read touches all 16 entries of the public group and keeps one with masks.
- The secret shift rotates columns, and moves rows in five steps of 1, 2, 4, 8 and 16, each kept or dropped with a mask.
- The shuffles are a bitonic sorting network that carries each generation along with its tag, so W is built in sorted order without reading G at a secret position.
- The key check and the tag comparison run in full every time.

The multiplication in rule v5 must take the same time for any operands, which holds on current x86 and 64-bit ARM cores, and not on some older or embedded ones.

Agreement. Three vector files (the demo set, the recommended set and the key check) fix algorithm version 1, and the reference and C implementations reproduce all three byte for byte. Differential fuzzing found no disagreement between the C implementation and the reference: 1,000,000 building-block cases and 300 messages across 20 configurations, on both builds.

A command-line tool. For trying the cipher on files, `lce` encrypts and decrypts them with the recommended set (https://github.com/t3spe/life-cipher-runner). It runs on a third implementation, in Go, whose tests check every value in the three vector files and further known answers from the reference implementation, for associated data of a block or more, messages of several blocks, and other IV offsets. It follows the specification's constant-time requirements, but Go makes no promise about the machine code it generates, and this implementation hasn't been through the timing and Valgrind checks of Section 5.4.

Speed. Measured with AVX2 on one performance core of an Intel Core i5-1240P, a laptop processor (6 November 2026). Its efficiency cores run the same code at about half the speed.

- **Encryption** runs at about 55 MB/s at $R = 24$, and 82 MB/s at $R = 16$. Encryption speed doesn't depend on k.
- **Rule v5′s serial chain** costs about a third of the throughput: the earlier rule, which reads B through the fold alone, runs at 121 MB/s at $R = 16$ on the same core.
- **Key setup** for the recommended set takes about 112 ms in constant time (105–121 ms over five keys). It's dominated by the k sorts, the setup run of $1 + kT/16 = 11,251$ blocks, and building H and W.
- **Memory:** W takes $T \times 128 = 768{,}000$ bytes per key, and G as much again during setup.

For context, the test harness measured ChaCha20-Poly1305, from a mainstream library, at about 1 GB/s, on a machine shared with other work.

7 The challenge

We offer the cipher at five strengths, to measure attacks by the rounds they break:

Tier	R	Init and finish rounds	Data
1	4	8	`life-challenge-R4.bin`
2	8	16	`life-challenge-R8.bin`
3	12	24	`life-challenge-R12.bin`
4	16	32	`life-challenge-R16.bin`
5	24	48	`life-challenge-R24.bin` (the recommended set)

- **Keys:** every tier uses $T = 6,000$, $S = 16$, $k = 30$ and version 1, under its own secret key: a mined key that passes vetting, drawn independently of the others.
- **Data:** each file holds 1,024 messages. Each message has a random IV, no associated data, and 1,024 blocks of zero bytes as plaintext, so its ciphertext is the keystream itself: 2^{20} blocks, 128 MiB, of known plaintext per tier.
- **Where:** the files are archived at https://doi.org/10.5281/zenodo.24020075, together with `challenge.json`, which lists each file's SHA-256 and each tier's key commitment.
- **Commitments:** each tier's commitment is SHA - 256($salt \,\|\, K$), with a 32-byte random salt. The keys and salts will be published on 10 November 2028, two years after the challenge opens, so anyone can check that the keys didn't change. From then on, those keys are burned.
- **Claims:** a claimed key is checked with `challenge/verify.py`, which uses the reference implementation and requires the key to reproduce the tier's first message from its IV. Claims go to the repository's issue tracker, where the timestamp settles who was first. Distinguishers and partial results are welcome too, and anyone may publish an attack.

What we know. With this much data, cell persistence would show at $R = 2$, which isn't offered. At $R = 4$ its correlation is about $8 \cdot 10^{-6}$, and a four-standard-deviation detection under the approximate independent-sample model takes about 2^{28} blocks, hundreds of times what each tier provides. Chosen-IV differentials need chosen IVs, which the data doesn't offer. We know no key-recovery attack on any number of rounds. So even the 4-round tier is beyond every attack we have.

8 Conclusion

The Life Cipher shows that an AEAD can be built around the Game of Life, rule B3/S23 unchanged, with mined long-lived configurations as keys. The construction is conventional where it can be: a duplex, a Feistel-like reversible round, a keyed table of round keys, and constant-time code throughout. Measured against standard tools, the cipher passes every test we have at the recommended strength, and extrapolation of the measured cell persistence estimates an attack reach of about 7 of its 24 rounds within the stated data limit. That estimate and the factor-of-three margin remain heuristic.

That's evidence, not proof, and only of the attacks we thought of. The cipher's real test is outside analysis. The challenge offers concrete targets for it, the 4-round tier first.

References

[AFK+08] J.-P. Aumasson, S. Fischer, S. Khazaei, W. Meier and C. Rechberger. New Features of Latin Dances: Analysis of Salsa, ChaCha, and Rumba. FSE 2008, LNCS 5086, pp. 470–488. Springer, 2008.

[BCG82] E. R. Berlekamp, J. H. Conway and R. K. Guy. Winning Ways for Your Mathematical Plays, Volume 2, Chapter 25, "What Is Life?". Academic Press, 1982.

[BDPV11] G. Bertoni, J. Daemen, M. Peeters and G. Van Assche. Duplexing the Sponge: Single-Pass Authenticated Encryption and Other Applications. SAC 2011, LNCS 7118, pp. 320–337. Springer, 2012.

[BK09] A. Biryukov and D. Khovratovich. Related-Key Cryptanalysis of the Full AES-192 and AES-256. ASIACRYPT 2009, LNCS 5912, pp. 1–18. Springer, 2009.

[BMP97] S. R. Blackburn, S. Murphy and K. G. Paterson. Comments on "Theory and Applications of Cellular Automata in Cryptography". IEEE Transactions on Computers 46(5), pp. 637–638, May 1997. Comment only; the combined comment-and-reply record spans pp. 637–639. https://doi.org/10.1109/12.589245.

[Dae95] J. Daemen. Cipher and Hash Function Design Strategies Based on Linear and Differential Cryptanalysis. PhD thesis, Katholieke Universiteit Leuven, 1995.

[DMV17] J. Daemen, B. Mennink and G. Van Assche. Full-State Keyed Duplex with Built-In Multi-user Support. ASIACRYPT 2017, LNCS 10625, pp. 606–637. Springer, 2017.

[Dot] C. Doty-Humphrey. PractRand, version 0.96. https://pracrand.sourceforge.net/

[GS] F. Pigorsch. GoL-SAT: a SAT-based forward/backwards solver for Conway's Game of Life. https://github.com/flopp/gol-sat

[Gar70] M. Gardner. Mathematical Games: The fantastic combinations of John Conway's new solitaire game "life". Scientific American 223(4), pp. 120–123, October 1970.

[Lan10] A. Langley. Checking that functions are constant time with Valgrind. ImperialViolet, 1 April 2010. https://www.imperialviolet.org/2010/04/01/ctgrind.html. Code: https://github.com/agl/ctgrind.

[LLS] O. Cunningham. Logic Life Search, a SAT-based search program for Life patterns. https://gitlab.com/OscarCunningham/logic-life-search. Overview: https://conwaylife.com/wiki/Logic_Life_Search.

[LS07] P. L'Ecuyer and R. Simard. TestU01: A C Library for Empirical Testing of Random Number Generators. ACM Transactions on Mathematical Software 33(4), Article 22, 2007.

[LW] LifeWiki. R-pentomino. https://conwaylife.com/wiki/R-pentomino (accessed 6 November 2026).

[MMB11] M. J. Machicao, A. G. Marco and O. M. Bruno. Chaotic Encryption Method Based on Life-Like Cellular Automata. arXiv:1112.6326, 2011.

[MRV15] B. Mennink, R. Reyhanitabar and D. Vizár. Security of Full-State Keyed Sponge and Duplex: Applications to Authenticated Encryption. ASIACRYPT 2015, LNCS 9453, pp. 465–489. Springer, 2015.

[MS91] W. Meier and O. Staffelbach. Analysis of Pseudo Random Sequences Generated by Cellular Automata. EUROCRYPT '91, LNCS 547, pp. 186–199. Springer, 1991.

[NIST25] NIST. Ascon-Based Lightweight Cryptography Standards for Constrained Devices: Authenticated Encryption, Hash, and Extendable Output Functions. Special Publication 800-232, August 2025.

[NKP94] S. Nandi, B. K. Kar and P. Pal Chaudhuri. Theory and Applications of Cellular Automata in Cryptography. IEEE Transactions on Computers 43(12), pp. 1346–1357, 1994.

[RBV17] O. Reparaz, J. Balasch and I. Verbauwhede. Dude, is my code constant time? DATE 2017, pp. 1697–1702. IEEE, 2017.

[RFC8439] Y. Nir and A. Langley. ChaCha20 and Poly1305 for IETF Protocols. RFC 8439, June 2018.

[Sch94] B. Schneier. Description of a New Variable-Length Key, 64-Bit Block Cipher (Blowfish). Fast Software Encryption 1993, LNCS 809, pp. 191–204. Springer, 1994.

[Sch98] B. Schneier. Memo to the Amateur Cipher Designer. Crypto-Gram, 15 October 1998. https://www.schneier.com/crypto-gram/archives/1998/1015.html

[TM87] T. Toffoli and N. Margolus. Cellular Automata Machines: A New Environment for Modeling. MIT Press, 1987.

[Wol86a] S. Wolfram. Cryptography with Cellular Automata. CRYPTO '85, LNCS 218, pp. 429–432. Springer, 1986.

[Wol86b] S. Wolfram. Random Sequence Generation by Cellular Automata. Advances in Applied Mathematics 7(2), pp. 123–169, 1986.